

这是已分享的 ChatGPT 对话副本
举报对话

昨天 9:09

上传文件

我想知道这种联想性mem或者说situation而不是简单的sim
做检索召回，还有没有类似的memory文章  深度研究

研究完成情况：12m · 30 次引用

联想性记忆与情境化检索召回研究报告



联想性记忆与情境化检索召回研究报告

行摘要

本报告聚焦于长时对话 / **Agent memory** 语境下的“联想性 memory / situation-based 检索召回”，而不是一
意义上的长上下文或传统文档 RAG。就目前公开文献看，这一方向并没有完全统一的命名；最接近的表述
有三类：**associative recall**、**cue-trigger semantic disconnect**、以及更工程化的**state-aware / graph**
aided / deliberate retrieval。你上传的 T-Mem 正是当前最直接命中这一问题定义的一手材料：它明确把
方向分为 **descriptive recall** 与 **associative recall** 两类，并指出现有多数系统虽然结构多样，但读时仍主
单一相似空间的 top-k 检索。^{1 2}

心结论可以先概括为一句话：“相似度召回”关心 query 与 memory 是否“像”，而“联想性 / situation-
sed 召回”关心 query 所处的目标、状态、因果链、时间演化与未来情境是否“相关”。前者更像“找近邻”，

昨天 13:11

我主要是在gui agent上做这些，可是我想知道这种情景
或者说超图，适合在gui的数据集上做吗？我看家好像都是
对话？这个idea是可行的吗？帮我调研一下mem-w之类的
gui agent数据集，然后从真实的样例中找到这个idea的可
行性

思考了 5m 3s >

结论

这个 idea 是可行的，而且 GUI Agent 实际上比长对话更适合做 situation-based / associative memory。

但论文的核心不能只是：

把 GUI 轨迹放进超图，再做图检索。

因为已有 HyMEM 已经在 GUI Agent 中使用“符号策略节点 + 连续轨迹表示 + 图扩展检索”；而且 T-Mem 也明确指出：使用图或超图，不代表已经摆脱 **similarity-first retrieval**。很多图记忆仍然先用 embedding 找到相似节点，再在附近扩展，本质上仍受初始相似度限制。

24 T-Mem(3)

你更有价值的定位应该是：

不是按照任务文字或轨迹内容检索，而是按照当前 GUI situation 检索；检索键是 situation，注入模型的证据仍然是原始截图、ROI、动作结果或 latent trajectory。

即你之前的表述：

Indexing by Situation, Injecting by Evidence

这个思路放在 GUI Agent 上是成立的，而且从现有数据的真实案例看，需求非常明确。

一、为什么 GUI 比对话更适合 situation memory

对话中的 situation 往往比较隐式，例如：

- 当前用户的价值观；
- 一个过去承诺是否仍然有效；
- 当前话题与过去事件之间的因果关系。

GUI 中的 situation 则可以被更明确地结构化：

$Situation_t = (\text{Goal}, \text{App}, \text{Page State}, \text{Subgoal}, \text{Constraints}, \text{Known Facts}, \text{Fail})$

例如：

- 已经在地图中找到了目的地；
- 当前切换到了打车软件；
- 还没有把目的地输入进去；
- 目的地名称必须从前一个应用传递过来。

当前 Lyft 页面在视觉上可能与过去的 Google Maps 页面完全不相似，当前动作指令也未必重新提及目的地名称。但是二者处于同一个跨应用信息交接 situation。

因此 GUI 里的核心关系不是：

当前截图和哪张历史截图最相似？

而是：

当前任务处于什么阶段？过去哪次经验解决过相同的依赖关系？

这比对话中的 situation 更容易定义、标注和验证，因为 GUI 有明确的：

- 页面变化；
- 应用切换；
- 动作结果；
- 子任务完成状态；
- 成功或失败 outcome。

二、现有 GUI memory 方法已经证明“需要 memory”，但尚未完全解决 situation retrieval

1. Mem-W：memory 表示很先进，但检索仍是 similarity-first

Mem-W 把两种历史统一压缩成 latent soft tokens：

- **Experiential memory**：从外部 trajectory bank 检索出的历史轨迹；
- **Working memory**：当前 episode 中已经被窗口淘汰的前缀。

二者使用同一个 Q-Former compressor，最终变成可以直接送入冻结 GUI Agent 的连续 token。

Mem-W

但是，Mem-W 的外部经验检索仍然是：

$$I_t = \text{TopM}_i \text{sim}(q_t, k_i)$$

而且实际实验中使用 FAISS，以归一化 embedding 的 inner-product similarity 检索 top-5 历史经验。

Mem-W

Mem-W

所以 Mem-W 解决的是：

检索到轨迹之后，如何高效、保真地压缩和注入。

而你的 situation memory 要解决的是它之前的另一个问题：

到底应该检索哪一段历史轨迹。

二者是正交的，甚至可以直接组合：

Situation Retriever → Retrieved Trajectory → Mem-W Compressor → Late

2. HyMEM: 已经开始做图关联, 但入口仍依赖相似度

HyMEM 是目前和你的想法最接近的 GUI 工作。它的每个 trajectory node 包括:

- high-level strategy, 例如“按价格从低到高筛选”;
- middle-level attributes, 例如 #search、#filter、\$price;
- continuous trajectory embeddings。

具有相同属性的节点建立边, 从而支持多跳图扩展。  arXiv

但它的初始 seed node 仍然来自:

$$[\text{CLIP}_{\text{text}}(q); \text{CLIP}_{\text{image}}(o_1)] \xrightarrow{\text{FAISS cosine}} \text{Top-K seeds}$$

随后才从 seed 节点向一跳邻居扩展。也就是说, 图确实能找到“不那么相似但相关”的邻居, 但前提是初始 similarity retrieval 没有走错区域。  arXiv

这给你的工作留下了很明确的空间:

HyMEM 是 **similarity seed + graph expansion**;
你的方法可以是 **situation trigger + evidence retrieval**。

3. MementoGUI: 证明了“事件与状态”是 GUI memory 的合理单位

MementoGUI 不再机械保存所有截图, 而是让 memory controller:

- 选择真正影响未来决策的 GUI event;
- 保存文本摘要和 ROI 视觉证据;
- 判断何时应该调用 episodic memory;
- 使用 learned relevance selector 选择历史轨迹。

它还专门提出 MementoGUI-Bench, 评估 task progress 和 memory consistency, 而不只是下一步 action accuracy。  arXiv

这已经提供了重要证据:

GUI memory 的合理单位不是固定 screenshot chunk, 而是会影响未来决策的 **event / state transition**。

但它主要研究“何时写、写什么、何时调用”, 并没有像 T-Mem 那样明确建立:

当前 situation → 未来使用 trigger → 历史证据

因此你的 situation indexing 仍然有独立价值。

三、从真实 GUI 样例看，这个问题确实存在

案例一：LinkedIn → Microsoft Word

GUI Odyssey-CoM 中的真实任务是：

在 LinkedIn 中寻找 mobile app developer 职位，然后在 Microsoft Word 中记录公司名称。

在 LinkedIn 页面上，系统发现：

Company Name: Apex Systems

随后需要回到 Word 中输入这个名称。CoM 将 Apex Systems 从 screen information 提取到 LTM，再跨应用使用。 2506.18158v1

这里真正的 memory situation 是：

```
source_app: LinkedIn
acquired_entity: Apex Systems
entity_role: company name
target_app: Microsoft Word
pending_subgoal: record acquired entity
```

当前 Word 页面与历史 LinkedIn 页面：

- 视觉上完全不同；
- UI element 完全不同；
- 动作也不同；
- 应用类别也不同。

但二者具有相同的信息交接关系。

适合的超边

$e_{\text{handoff}} = \{\text{LinkedIn result, Apex Systems, Word document, record-company name}\}$

这不是一组任意 pairwise edges，而是四个元素同时成立时才构成完整 situation，因此超图在这里是有意义的。

案例二：Waze → Uber

CoM 的另一个真实任务是：

用 Waze 找附近的健身房，然后用 Uber 前往。

Waze 找到：

Trojan Crossfit, 780 米, 4.8 星

之后进入 Uber 时, Agent 必须记住并使用 Trojan Crossfit 。 2506.18158v1

把它和前面的 LinkedIn→Word 放在一起看：

表面任务	来源应用	目标应用	需要传递的内容
找工作并记录	LinkedIn	Word	公司名称
找健身房并打车	Waze	Uber	地点名称

如果按照 task embedding，它们并不相似：

- 一个是求职；
- 一个是导航；
- 一个是文本编辑；
- 一个是打车。

但按照 situation，它们完全相同：

在应用 A 获取一个 entity，切换到应用 B 后消费这个 entity。

这就是 GUI 上最典型的 associative memory。

案例三：Google Maps → Lyft

Mem-W 第 28 页的真实案例是：

用 Google Maps 找最近的书店，然后通过 Lyft 打车过去。

无 memory 的 agent 在多个步骤中出现错误和重复；Mem-W 则能够正确维护进度、返回主屏幕、进入后续应用并继续任务。 Mem-W

这与 Waze→Uber 的表面内容不同：

- Waze vs Google Maps；
- Uber vs Lyft；
- 健身房 vs 书店。

但 situation 几乎完全相同：

```

Map/Search app
  ↓ discovers destination
Destination entity
  ↓ must survive app switch
Ride-hailing app
  ↓ consumes destination
Book trip

```

这给你提供了非常强的真实数据对：

- **Memory evidence**: Waze→Uber 的历史轨迹；
- **Current query**: Google Maps→Lyft；
- 文本和界面不完全相同；
- 底层 situation 相同。

这可以直接用于构造 GUI 版本的 LoCoMo-Plus 式 associative evaluation。

案例四：安装 Triller → Settings → 重开 Triller

Mem-W 的另一个真实任务是：

1. 在 Google Play 安装 Triller；
2. 打开 Triller；
3. 去 Settings 关闭 Triller 通知；
4. 重新打开 Triller 看视频。

Mem-W 成功维护了整个跨应用阶段；vanilla agent 在应用切换和 Settings 中发生错误。

Mem-W

Mem-W 检索到的两条历史轨迹几乎是该任务的近重复版本。

Mem-W

这恰好暴露了当前方法的局限：

它证明了相似轨迹 retrieval 有帮助，但还没有证明能否迁移到“不相似但 situation 相同”的任务。

一个更能验证你方法的测试可以是：

History:
安装 Triller → 在 Settings 关闭通知 → 重开 Triller

Test:
安装另一个应用 → 在系统设置修改该应用权限 → 重开应用

二者的 app 名称、图标和文字不同，但 situation 都是：

Install → Open → System-level configuration → Return to app

这里应检索的是 **system-setting detour pattern**，而不是“Triller”这个实体。

需要明确：后一个任务是基于真实轨迹设计的 counterfactual evaluation，不是原数据集中已经报告的样本。

四、哪些 GUI 数据集最适合做

GUI Odyssey / GUI Odyssey-CoM：最适合跨应用 situation

GUI Odyssey-CoM 基于 GUI Odyssey 的 7,735 个 episode、201 个 app，平均约 15.4 步，并增加了 screen information、action description、reasoning 和 long-term memory 标注；总计约 111,791 个 screen-action pair。

2506.18158v1

2506.18158v1

它尤其适合：

- source app → target app 信息交接；
- app switch 后的子任务恢复；
- completed / pending subgoal 建模；
- 跨 app entity dependency。

这是最适合你第一阶段实验的数据集。

Mem-W 的 web/mobile trajectory bank：适合经验检索训练

Mem-W 使用的数据包括：

- Web：13 个 domain，11,176 条成功训练轨迹，22,346 条成功 memory bank 轨迹；
- Mobile：6 类任务，约 2,489 条成功训练轨迹，4,972 条成功 memory bank 轨迹。

Mem-W

Mem-W

它们适合训练：

- situation encoder；
- trajectory outcome encoder；
- success/failure-aware evidence compressor；
- situation-to-trajectory alignment。

但它们原本没有 situation label，需要由规则或 MLLM 从轨迹中重新抽取。

Multimodal-Mind2Web / MMINA：适合跨网站策略迁移

Multimodal-Mind2Web 包含超过 2,000 个开放任务、137 个真实网站、31 个 domain；MMINA 则包含多跳、多模态网络任务。 Mem-W

适合研究：

- 相同 workflow 在不同网站布局下的迁移；
- delayed constraints；
- 搜索—筛选—核验—提交；
- 同一 situation 下不同 UI appearance。

Agent Workflow Memory 已经证明，从历史轨迹中抽取 reusable workflow 能改善 Mind2Web 和 WebArena，说明 GUI 经验中确实存在跨任务可复用结构。  arXiv

不过 workflow 通常比较静态；它不能充分区分：

“当前任务整体属于什么 workflow”
与
“当前已经执行到 workflow 的哪个 situation”。

你的方法应重点解决后者。

MementoGUI-Bench：适合验证 state-aware memory

MementoGUI-Bench 专门面向：

- 延迟约束；
- 已完成子目标；
- 当前任务进度；
- 历史选择一致性；
- 需要回忆 ROI 视觉证据的决策。

因此它非常适合验证你的 working/situation memory，但它还不是专门针对“surface-disconnected associative retrieval”设计的。  arXiv

五、超图到底应该怎么建

我不建议把每张截图当作一个 node，然后把相邻动作连边。这种设计只是 trajectory graph，很难体现新意。

更合理的是两层结构。

1. Evidence layer

保存真正可供 policy 使用和审计的证据：

```
Screenshot / ROI
Action
Action Result
DOM / Accessibility information
Timestamp
Success / Failure outcome
```

2. Situation layer

保存用于检索的抽象状态：

```
Application
Task phase
Active subgoal
Completed subgoal
Pending dependency
Known entity
User constraint
UI state
Failure mode
Expected outcome
```

3. Hyperedge 类型

信息交接边

```
Handoff(
    source_app,
    acquired_entity,
    target_app,
    pending_subgoal
)
```

约束核验边

```
ConstraintCheck(
    candidate,
    price_limit,
    required_features,
    unmet_constraint,
    decision
)
```

阶段转换边

```

PhaseTransition(
    old_phase,
    transition_action,
    new_app_or_page,
    preserved_information
)

```

错误恢复边

```

Recovery(
    failed_state,
    failed_action,
    failure_cause,
    corrective_action,
    final_outcome
)

```

这样同一条轨迹可以属于多个 situation，而一个 situation 也可以关联多个不同任务的 evidence。超图在这里比普通图合理，因为真正有意义的是多个变量的**联合依赖**，而不是孤立的两两共现。

六、最重要的评测设计

现有 GUI benchmark 可以支持这个 idea，但直接在原 test split 上报告成功率，并不足以证明 **associative retrieval**。因为测试指令和 memory task 往往仍有明显词汇相似性。

你需要额外构造一个 **Situation-Disjoint / Surface-Disjoint split**：

Positive pair

底层 situation 相同，但：

- app 不同；
- entity 不同；
- screenshot 差异大；
- task wording 差异大。

例如：

```

Waze → Uber, 健身房
Google Maps → Lyft, 书店

```

Hard negative

task 文字或 app 相似，但当前 situation 不同。

例如：

同样都是 Google Maps，
一个还在搜索地点，
另一个已经找到地点、正在切换到打车软件。

后者对判断方法是否真正理解 situation 至关重要。

建议至少报告：

- **Evidence Recall@K**：关键历史证据是否被召回；
- **Situation Recall@K**：正确 situation 是否进入候选；
- **Task Success Rate**；
- **Cross-App Transfer Success**；
- **False Retrieval Rate**；
- **Memory Contribution Rate**：召回的 memory 是否实际帮助成功；
- **Token / latency cost**。

最终判断

这个方向不仅可行，而且现有工作正好形成了一条清晰的缺口：

- **CoM** 证明跨应用需要显式保存关键信息；
- **Mem-W** 证明 latent trajectory evidence 有效，但 retrieval 仍是 similarity top-K；
- **AWM** 证明 workflow 可以跨任务迁移；
- **HyMEM** 证明图结构与多跳关联有帮助，但初始入口仍依赖相似度；
- **MementoGUI** 证明 event、ROI、task state 是合理的 GUI memory 单位；
- 目前仍缺少一个系统明确解决：

当当前 GUI 与历史轨迹在任务文字、应用和视觉上都不相似时，如何根据共同的任务 situation 找到真正有用的历史证据。

因此最稳的论文定位不是“Hypergraph Memory for GUI Agents”，而是：

Situation-Indexed, Evidence-Grounded Memory for GUI Agents

超图是实现 situation 联合依赖的一种结构；真正的贡献是：

1. 将 **retrieval index** 与 **injected evidence** 解耦；
2. 用 GUI situation 而非 task similarity 建立可达性；
3. 从成功和失败轨迹中提取可迁移的 situation；
4. 审计“哪个 situation edge 召回了哪段 evidence，以及它是否帮助任务完成”。

这比单纯将 T-Mem 从对话迁移到 GUI 更有独立性，也比仅在 Mem-W 上替换一个 graph retriever 更完整。

 来源